SQL Trigger Exercises

Students work in pairs, switching the keyboard operator after each activity. Use a separate practice database.

1. Decode a trigger

Using the `record_tuition_change` trigger, answer the following questions:

1. Which table and operation activate the trigger?
2. Does it run before or after the operation?
3. Why is `FOR EACH ROW` required?
4. What is the difference between `OLD.tuition` and `NEW.tuition`?
5. Where did the values for `OLD` and `NEW` come from?
6. Why does the trigger check whether tuition changed?
7. Why is `DELIMITER` changed temporarily?

2. Predict and test tuition history

Begin with empty tables and insert:

```
INSERT INTO courses (course_id, course_name, tuition)
VALUES
    (101, 'Database Systems', 800.00),
    (102, 'Programming', 1000.00),
    (103, 'Networking', 600.00);
```

Before executing each statement below, predict how many history records it will create and what values they will contain.

```
-- A (how many entries added to tuition_history?)
UPDATE courses
SET tuition = 850.00
WHERE course_id = 101;

-- B (how many entries added to tuition_history?)
UPDATE courses
SET course_name = 'Introduction to Programming'
WHERE course_id = 102;

-- C (how many entries added to tuition_history?)
UPDATE courses
SET tuition = 850.00
WHERE course_id = 101;

-- D (how many entries added to tuition_history?)
UPDATE courses
SET tuition = tuition + 50.00;
```

Check the results:

```
SELECT * FROM tuition_history
```

3. Record payments automatically — 15 minutes

Create the document's `student_accounts` and `payments` tables and its `apply_student_payment` trigger. Start with empty tables.

```
INSERT INTO student_accounts
    (student_id, student_name, balance)
VALUES
    (1, 'Maria Garcia', 949.00),
    (2, 'James Lee', 1200.00);
```

Predict both balances, then execute:

```
INSERT INTO payments (student_id, amount)
VALUES
    (1, 200.00),
    (1, 100.00),
    (2, 300.00);
```

Check:

```
SELECT * FROM student_accounts
```

Answer:

1. How many times did the trigger run?
2. How did it identify the account to update?

4. Assume a table exists as follows: stuff(sid, description, mycount)

Complete the following **`BEFORE UPDATE`** trigger code so that if the new value in **`mycount`** is >= 100 for a particular **`sid`**, then it resets count to 0.

```
DELIMITER //
CREATE TRIGGER _______________________________
BEFORE UPDATE ON _____________________________
FOR EACH ROW
BEGIN









END //
DELIMITER ;
```